

2 Decks, shuffling

2.1 Decks

Each exercise simulates a game of solitaire played with 52 cards, using the following convention.

- Numbers 0 to 12 represent the Hearts, from Ace to King.
- Numbers 13 to 25 represent the Diamonds, from Ace to King.
- Numbers 26 to 38 represent the Clubs, from Ace to King.
- Numbers 39 to 51 represent the Spades, from Ace to King.

For example, 16 represents the Four of Diamonds, and 36 represents the Jack of Clubs.

2.2 Shuffling

Both exercises require shuffling a deck of cards according to the following convention. By *shuffling a deck of cards*, we mean randomly permuting the corresponding list of integers by passing the list, **arranged in increasing order**, as an argument to the `shuffle()` function from Python's `random` module. For instance, to shuffle the full 52-card deck, we can write:

```
>>> cards = list(range(52))
>>> shuffle(cards)
```

As another example, to shuffle the deck excluding the Four of Diamonds and the Jack of Clubs, we can write:

```
>>> cards = sorted(set(range(52)) - {16, 36})
>>> shuffle(cards)
```

To obtain reproducible results, the `seed()` function from the `random` module should be called with a specified argument immediately before invoking `shuffle()`.

By *shuffling the 52-card deck with 678 passed to `seed()`*, we mean performing the following steps:

```
>>> cards = list(range(52))
>>> seed(678)
>>> shuffle(cards)
```

This results in `cards` being equal to

```
[11, 12, 22, 38, 15, 16, 14, 28, 4, 34, 46, 48, 33,
 18, 5, 17, 27, 37, 50, 51, 31, 41, 9, 1, 39, 3,
 29, 40, 43, 23, 25, 13, 19, 35, 26, 42, 24, 32, 44,
 45, 6, 36, 8, 47, 2, 30, 10, 49, 21, 0, 20, 7]
```

3 First game of solitaire

3.1 Game description

4 First game of solitaire

4.1 Game description

The aim of the game is to remove the 12 picture cards (Jacks, Queens, and Kings). The game proceeds in rounds, each based on a layout of 16 cards drawn from the deck.

Initially, all 52 cards are placed face down. The top 16 cards are removed and placed face up in a 4×4 array, filled from left to right and from top to bottom, as shown in Figure 1. Thus, the top card of the deck occupies position 1, the next card position 2, and so on.

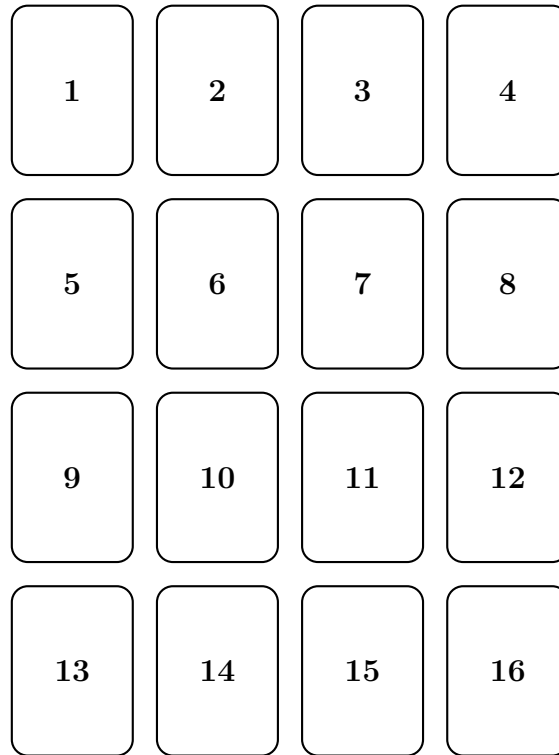


Figure 1: Layout positions for the 16 face-up cards (filled left-to-right, top-to-bottom).

All picture cards among these 16 cards are removed. If $n \geq 1$ picture cards are removed, then the top n cards from the remaining deck are drawn and placed into the vacant positions, again following the order indicated in Figure 1. This process is repeated until no picture card appears in the layout, at which point the round ends.

If all 12 picture cards have been removed, the game is won. Otherwise, the remaining face-up cards are

combined with the undealt cards to form a new deck of size $52 - k$, where k is the number of picture cards already removed. The deck is shuffled and a new round begins.

The game ends when either all 12 picture cards have been removed (in at most four rounds), in which case the game is won, or after the fourth round if at least one picture card remains, in which case the game is lost.

4.2 Playing a single game (3.5 marks)

Your program must be stored in a file named `solitaire_1.py`. Executing

```
$ python3 solitaire_1.py
```

at the Unix prompt must produce the following output (ending with a single space):

```
Enter an integer to pass to the seed() function:
```

The program then waits for user input, which may be assumed to be an integer. That integer is passed to `seed()` before calling `shuffle()`, as described in Section 2.

Shuffling and rounds. Each round begins with a shuffle. The first shuffle occurs before the first round. If further rounds are needed (up to four in total), a new shuffle is performed before each round using the remaining cards.

Before every shuffle:

- the cards to be shuffled must be sorted;
- the value passed to `seed()` is increased by 1.

Thus, if the user inputs 275:

- before the first round, `seed(275)` is called;
- before the second round (if any), `seed(276)` is called;
- before the third round (if any), `seed(277)` is called;
- before the fourth round (if any), `seed(278)` is called.

Initial output. The output begins with an empty line followed by:

```
Deck shuffled. Ready to start!
```

The next line represents the deck with all cards facing down (a sequence of 52 `]` characters).

Round announcements. Each round announcement is preceded by exactly one empty line.

The first round is announced by:

Starting the first round...

Subsequent rounds (if any) are announced by:

After shuffling, starting the _ round...

where _ is `second`, `third`, or `fourth`.

Structure of a round. Each round consists of one or more blocks. Each block has the following structure:

- An empty line.
- A line of the form

Drawing _ card[s]:

where _ is the number of cards drawn (16 at the start of a round, and thereafter equal to the number of picture cards just removed.). Use `card` if the number is 1 and `cards` otherwise.

- One line showing the current deck (a sequence of `]` characters).
- Four lines representing the 4 rows of face-up cards.

If no picture card is removed, the block ends here.

If $i \geq 1$ picture cards are removed, the block continues with:

- An empty line.
- A line of the form

Removing _ picture card[s]:

where _ is i (use `picture card` if $i = 1$ and `picture cards` otherwise).

- Four lines showing the updated 4×4 layout after removing the pictures.

End of the game. The final output ends with an empty line followed by exactly one of:

You removed all picture cards. You won! 😊

You removed no picture cards. You lost! 😞

You removed only _ picture card[s]. You lost! 😞

where `_` is an integer between 1 and 11 (with correct singular/plural agreement).

An example interaction for a game that is lost is provided in [solitaire_1_1.pdf](#).

An example interaction for a game that is won is provided in [solitaire_1_2.pdf](#).

Formatting requirements.

- Face-up cards are displayed using the appropriate Unicode character.
- Within each row, the first card is preceded by one tab character, the second by two tabs, the third by three tabs, and the fourth by four tabs.
- No output line contains trailing spaces.

4.3 Playing many games and estimating probabilities (3 marks)

Running

```
$ python3
```

at the Unix prompt, and then

```
>>> from solitaire_1 import simulate
```

at the Python prompt, allows you to call the `simulate()` function, which takes two arguments.

- The first argument, say n , is a strictly positive integer (you may assume it is valid). It represents the number of games to simulate.
- The second argument, say i , is an integer (you may assume it is valid). It determines the initial seed used for the simulations.

The function simulates n independent games. For the g^{th} game ($g = 0, 1, \dots, n-1$), shuffles are performed exactly as in Section 2, except that the value passed to `seed()` is determined as follows:

- Before the first round of game g , `seed()` is called with $i + g$.
- Within the same game, before each subsequent round (if any), the seed is increased by 1.

Equivalently, if game g reaches round $r \in \{0, 1, 2, 3\}$ (with $r = 0$ for the first round), then the shuffle before that round uses `seed(i+g+r)`.

An example interaction is provided in [simulate_1_1.pdf](#).

A further example interaction is provided in [simulate_1_2.pdf](#).

Output format. The function outputs a table estimating the probabilities of removing k picture cards, for $k \in \{0, 1, \dots, 12\}$, based on the n simulated games.

The output begins with the two lines:

```
Number of picture cards removed | Relative frequency
```

```
-----
```

Each subsequent line corresponds to a value of k with strictly positive estimated probability. Lines are printed in increasing order of k .

Probabilities are computed as floating-point relative frequencies and displayed as percentages with exactly two digits after the decimal point.

Only strictly positive probabilities and the corresponding numbers of uncovered picture cards are printed. This includes probabilities that are smaller than 0.005%, and which therefore appear as 0.00%.

Each data line has the form

```
<right-aligned k> | <right-aligned percentage>
```

There is exactly one space on each side of the vertical bar. All lines in the table (including the header and separator) consist of exactly 52 characters.

5 Second game of solitaire

5.1 Game description

The aim of the game is to place all 52 cards onto 8 stacks arranged in two rows, corresponding (from left to right) to Hearts, Diamonds, Clubs and Spades.

- The top row consists of increasing sequences built from Aces upward (Ace, 2, 3, ..., King).
- The bottom row consists of decreasing sequences built from Kings downward (King, Queen, Jack, ..., Ace).

Each suit therefore defines two possible stacks:

- an increasing stack that must begin with the Ace of that suit,
- a decreasing stack that must begin with the King of that suit.

A stack is created only when its base card (Ace or King) is first placed. Once created, a stack may be extended only by placing on top of it the next card of the same suit in the required direction.

Initial configuration. All 52 cards are initially placed face down in a single deck. The first card of the deck is at the bottom, and the last card is at the top.

Cards are drawn from the top of the deck in groups of three, except possibly for the final group, which may consist of one or two cards if fewer than three cards remain.

When cards are drawn, they are placed face up, one on top of another, in the order in which they are removed from the deck. Thus, the last card drawn is the topmost face-up card.

Only the topmost face-up card is eligible for placement. If that card is placed, the card immediately beneath it becomes eligible. This continues until no face-up card can be placed.

Whenever no face-up card can be placed and the deck is not empty, up to three additional cards are drawn from the deck and placed face up on top of the existing face-up cards.

Placing cards. A card may be placed if and only if one of the following applies:

- it is an Ace or a King, in which case it starts the corresponding stack;
- it can be placed on top of an existing increasing stack of the same suit;
- it can be placed on top of an existing decreasing stack of the same suit.

Whenever a card can be placed, it is immediately removed from the face-up pile and added to the appropriate stack.

If the topmost face-up card cannot be placed, it remains face up and blocks access to the cards beneath it in the pile.

Priority rules. If a card can either start a new stack or extend an existing stack, it must start the new stack.

If a card can extend both an increasing and a decreasing stack of the same suit, it must extend the increasing stack.

Continuation of a round. The process continues until the deck becomes empty

At the end of a round:

- if no card has been placed during that round, the game is lost;
- otherwise, all unplaced face-up cards are collected, turned face down (preserving their order), and form a new deck for the next round.

A new round then begins.

End of the game. The game ends when either:

- all 52 cards have been placed onto the 8 stacks, in which case the game is won;
- a round completes without any card being placed, in which case the game is lost.

5.2 Playing a single game (3.5 marks)

Your program must be stored in a file named `solitaire_2.py`. Executing

```
$ python3 solitaire_2.py
```

at the Unix prompt must produce the following output (ending with a single space):

```
Enter an integer to pass to the seed() function:
```

The program then waits for user input, which may be assumed to be an integer. That integer is passed to `seed()` before calling `shuffle()`, as described in Section 2.

Initial output. The output begins with an empty line followed by exactly one of:

```
You placed all cards. You won! 😊
```

or

```
You could not place _ cards. You lost! 😞
```

where `_` is an integer between 3 and 52.

Collected output. All lines produced during the simulation are stored as collected output.

The first line of the collected output is:

```
Deck shuffled. Ready to start!
```

The second line represents the full deck with all cards facing down (a sequence of 52 `]` characters).

Structure of the collected output. The key pattern of the collected output consists of four lines. Successive occurrences of this pattern are separated by exactly one empty line.

- The first line represents the current deck. It consists of as many `]` characters as there are cards remaining in the deck. When the deck is empty, this line is empty.
- The second line represents the face-up cards that have been taken from the deck but not placed. The topmost such card is displayed using the appropriate Unicode character; any cards beneath it are represented by `[` characters. If no such cards exist, this line is empty.

- The third line represents the increasing stacks (built on Aces). When not empty, the line begins with four spaces. The line contains four possible positions corresponding (from left to right) to Hearts, Diamonds, Clubs and Spades. The starts of these positions are separated by 15 characters. For each suit whose stack is nonempty, the top card of the stack is displayed using its Unicode character; any cards beneath it are represented by [characters.
- The fourth line represents the decreasing stacks (built on Kings), following the same formatting rules as the third line.

No line contains any trailing space, and no tab character appears anywhere in the collected output.

Announcements during play. At the beginning of each round, the following line is added to the collected output. It is preceded by exactly one empty line and followed by exactly one empty line:

Starting the _ round...

where _ is one of: 1st, 2nd, 3rd, 4th, 5th, ...

Post-game interaction. After the initial win/loss message, the program outputs an empty line followed by:

There are _ lines of output. What do you want me to do?

Enter: q to quit

a last line number (between 1 and _)

a first line number (between -1 and -_)

a range of line numbers (of the form m--n with $1 \leq m \leq n \leq _$)

All occurrences of _ denote the same integer.

The program then waits for user input on the next line. Before reading the input, the program outputs spaces so that the user's input begins aligned under the letter q and the three leftmost as of the prompt above.

The interaction then proceeds as follows. Until q is entered:

- an empty line is printed;
- if the input is correct, the requested portion of the collected output is displayed;
- an empty line is printed;
- the prompt is displayed again and the program waits for input aligned as described above.

The program exits when q is entered.

An input is considered correct only if it exactly matches one of the required forms, with integers in the specified ranges (positive integers must not be preceded by +). Arbitrary spaces are permitted:

- at the beginning or end of the input,
- before the first - in a range,
- after the second - in a range.

No space is allowed between the minus sign and the digits of a negative number.

An example interaction with several incorrect user inputs is provided in [solitaire_2_1.pdf](#).

An example interaction displaying the complete collected output for a game that is lost is provided in [solitaire_2_2.pdf](#).

[solitaire_2_3.pdf](#) shows an example interaction that displays the complete collected output for a game that is won.

5.3 Playing many games and estimating probabilities (3 marks)

Running

```
$ python3
```

at the Unix prompt, and then

```
>>> from solitaire_2 import simulate
```

at the Python prompt, allows you to call the `simulate()` function, which takes two arguments.

- The first argument, say n , is a strictly positive integer (you may assume it is valid). It represents the number of games to simulate.
- The second argument, say i , is an integer (you may assume it is valid). It determines the initial seed used for the simulations.

The function simulates n independent games. For the g^{th} game ($g = 0, 1, \dots, n - 1$), the deck of 52 cards is shuffled exactly as in Section 2, using `seed(i+g)` before calling `shuffle()`.

An example interaction is provided in [simulate_2_1.pdf](#).

A further example interaction is provided in [simulate_2_2.pdf](#).

Output format. The function outputs a table estimating the probabilities that a game ends with k cards left unplaced, for values of k observed in the n simulated games.

Probabilities are computed as relative frequencies and displayed as percentages with exactly two digits after the decimal point.

Only values of k with strictly positive estimated probability are printed, including values whose percentage rounds to `0.00%`.

Lines are printed in decreasing order of k .

The output begins with the two lines:

```
Number of cards left | Relative frequency
-----
```

Each subsequent line corresponds to a printed value of k and has the form:

```
<right-aligned k> | <right-aligned percentage>
```

There is exactly one space on each side of the vertical bar.

All lines in the table (including the header and separator) consist of exactly 41 characters.